

Regular Expression — Pattern দিয়ে Text খোঁজা, যাচাই, Extract ও Replace করা

Chapter 13-এ আমরা RegExp-এর প্রাথমিক ধারণা দেখেছি। প্রফেশনাল সফটওয়্যার ইঞ্জিনিয়ারিংয়ে ব্যবহারকারীর ইনপুট ভ্যালিডেশন, টেক্সট ক্লিনিং, ইউজারনেম/ইমেইল/পাসওয়ার্ড ভেরিফিকেশন এবং সিকিউর স্যানিটাইজেশনের জন্য **Advanced Regular Expressions** একটি অত্যন্ত শক্তিশালী টুল।

Text String → RegExp Engine (/pattern/flags) → Token Matching / Capturing Groups → Extraction / Validation / Replacement

সরাসরি Form Validation, API Data Cleaning এবং Text Analytics-এর শতভাগ নিখুঁত সমাধান তৈরি করে এই রেজেক্স আর্কিটেকচার। 🔥

Part 01 — RegExp Core Mechanics, Engine Construction & Flags (23.1 – 23.8)

23.1 Regular Expression কী?

Regular Expression বা Regexp হলো একটি pattern language।

সহজভাবে:

কোনো text-এর মধ্যে নির্দিষ্ট ধরনের pattern খোঁজা বা যাচাই করার নিয়ম হলো Regular Expression।

ধরো:

OUTPUT / TEXT

Hello Shariar
Hello Rahim
Hello Karim

তুমি যদি এমন text খুঁজতে চাও যেটা:

OUTPUT / TEXT

Hello + একটি নাম

তাহলে Regex pattern ব্যবহার করতে পারো।

23.2 Real-Life Example

ধরো একটি registration form:

OUTPUT / TEXT

Email: _____
Phone: _____
Password: _____

User লিখল:

OUTPUT / TEXT

abc@xyz.com

তুমি check করতে চাও:

এটা email-এর মতো দেখতে কি না?

Regex দিয়ে pattern check করা যায়।

23.3 Regex তৈরি করার দুইটি উপায়

1. Regex Literal

23.3 JAVASCRIPT REGEXP

ES2026

```
const pattern = /hello/;
```

2. RegExp Constructor

23.3 JAVASCRIPT REGEXP

ES2026

```
const pattern = new RegExp("hello");
```

সাধারণ static pattern-এর জন্য literal বেশি convenient।

23.4 test()

Pattern match করছে কি না দেখতে:

23.4 JAVASCRIPT REGEXP

ES2026

```
const pattern = /hello/;  
  
console.log(pattern.test("hello world"));
```

CONSOLE OUTPUT

true

আরেকটি:

23.4 JAVASCRIPT REGEXP

ES2026

```
const pattern = /hello/;  
  
console.log(pattern.test("Hi world"));
```

OUTPUT / TEXT

false

23.5 Case Sensitivity

23.5 JAVASCRIPT REGEXP

ES2026

```
const pattern = /hello/;  
console.log(pattern.test("Hello"));
```

OUTPUT / TEXT

false

কারণ:

OUTPUT / TEXT

hello

এবং:

OUTPUT / TEXT

Hello

এক নয়।

23.6 i` Flag

Case-insensitive করতে:

23.6 JAVASCRIPT REGEXP

ES2026

```
const pattern = /hello/i;  
console.log(pattern.test("Hello"));
```

OUTPUT / TEXT

true

এখন:

OUTPUT / TEXT

hello
HELLO
Hello
hELLO

সব match করবে।

23.7 গুরুত্বপূর্ণ Regex Flags

Flag	কাজ
i	Case-insensitive
g	Global search
m	Multiline
s	Dot যেন newline-ও match করতে পারে
u	Unicode-aware matching
y	Sticky matching
d	Match indices তৈরি করতে সাহায্য করে

Example:

23.7 JAVASCRIPT REGEXP

ES2026

```
/hello/gi
```

মানে:

```
g → সব match  
i → case-insensitive
```

23.8 g`Flag

23.8 JAVASCRIPT REGEXP

ES2026

```
const text = "cat cat cat";  
  
console.log(  
  text.match(/cat/g)  
);
```

23.8 JAVASCRIPT REGEXP

ES2026

```
["cat", "cat", "cat"]
```

g না দিলে:

23.8 JAVASCRIPT REGEXP

ES2026

```
console.log(  
  text.match(/cat/)  
);
```

শুধু প্রথম match-এর information পাওয়া যায়।

Part 02 — Character Classes, Negation, Shorthands & Metacharacters (23.9 – 23.16)

23.9 Character Classes

Regex-এর সবচেয়ে গুরুত্বপূর্ণ অংশগুলোর একটি।

[abc]

মানে:

a, b অথবা c —যেকোনো একটি character।

23.9 JAVASCRIPT REGEXP

ES2026

```
console.log(/[abc]/.test("apple"));
```

OUTPUT / TEXT

true

কারণ apple -এ a আছে।

23.10 [a-z]

23.10 JAVASCRIPT REGEXP

ES2026

```
/[a-z]/
```

মানে:

OUTPUT / TEXT

a থেকে z

Example:

23.10 JAVASCRIPT REGEXP

ES2026

```
console.log(/[a-z]/.test("hello"));
```

OUTPUT / TEXT

true

23.11 [A-Z]

23.11 JAVASCRIPT REGEXP

ES2026

```
console.log(/[A-Z]/.test("Hello"));
```

OUTPUT / TEXT

true

23.12 [0-9]

Number character:

23.12 JAVASCRIPT REGEXP

ES2026

```
console.log(/[0-9]/.test("abc5"));
```

OUTPUT / TEXT

true

23.13 Negated Character Class

[^ ...]

মানে:

এই characters-এর কোনোটি নয়।

Example:

23.13 JAVASCRIPT REGEXP

ES2026

```
/[^0-9]/
```

মানে:

digit নয় এমন character।

23.13 JAVASCRIPT REGEXP

ES2026

```
console.log(/[^0-9]/.test("123"));
```

OUTPUT / TEXT

false

কিন্তু:

23.13 JAVASCRIPT REGEXP

ES2026

```
console.log(/[^0-9]/.test("123a"));
```

OUTPUT / TEXT

true

কারণ a digit নয়।

23.14 Shorthand Character Classes

Regex-এ কিছু shortcut আছে।

`\d`

Digit:

OUTPUT / TEXT

0-9

23.14 JAVASCRIPT REGEXP

ES2026

```
console.log(/\d/.test("abc5"));
```

OUTPUT / TEXT

true

`\D`

Non-digit:

23.14 JAVASCRIPT REGEXP

ES2026

```
console.log(/\D/.test("123"));
```

OUTPUT / TEXT

false

`\w`

Word character:

সাপেক্ষণত:

OUTPUT / TEXT

A-Z

a-z

0-9

-

23.14 JAVASCRIPT REGEXP

ES2026

```
console.log(/\w/.test("hello"));
```

OUTPUT / TEXT

true

`\W`

Non-word character।

`\s`

Whitespace:

OUTPUT / TEXT

space

tab

newline

`\S`

Non-whitespace।

23.15 Dot `.

Regex-এ:

23.15 REGEXP PATTERN

PATTERN

.

সাধারণত newline ছাড়া যেকোনো একটি character match করে।

23.15 JAVASCRIPT REGEXP

ES2026

```
console.log(/c.t/.test("cat"));
```

OUTPUT / TEXT

true

এগুলোও match করতে পারে:

OUTPUT / TEXT

cat
cot
cut
c9t

23.16 Dot + s Flag

s flag ব্যবহার করলে dot newline-ও match করতে পারে।

23.16 JAVASCRIPT REGEXP

ES2026

```
const text = "Hello\nWorld";  
  
console.log(/Hello.World/s.test(text));
```

OUTPUT / TEXT

true

Part 03 — Quantifiers, Boundary Anchors & Alternation Patterns (23.17 – 23.28)

23.17 Quantifiers

Quantifier বলে:

কোনো pattern কতবার থাকতে পারবে।

সবচেয়ে গুরুত্বপূর্ণ:

OUTPUT / TEXT

+
*
?
{n}
{n,m}
{n,}

23.18 +

+ মানে:

এক বা তার বেশি।

23.18 JAVASCRIPT REGEXP

ES2026

```
console.log(/\d+/.test("12345"));
```

OUTPUT / TEXT

true

+ ছাড়া:

23.18 JAVASCRIPT REGEXP

ES2026

/\d/

শুধু একটি digit match করার concept।

23.19

* মানে:

zero বা তার বেশি।

23.19 JAVASCRIPT REGEXP

ES2026

```
console.log(/go*/.test("g"));
```

OUTPUT / TEXT

```
true
```

কারণ `o` zero times-ও থাকতে পারে।

এগুলো match করতে পারে:

OUTPUT / TEXT

```
g
go
goo
gooo
```

23.20 ?

? মানে:

zero অথবা one time।

23.20 JAVASCRIPT REGEXP

ES2026

```
/go?/.test("go");
```

true।

23.20 JAVASCRIPT REGEXP

ES2026

```
/go?/.test("g");
```

এটিও true।

23.21 {n}

ঠিক `n` বার।

23.21 JAVASCRIPT REGEXP

ES2026

```
/\d{4}/
```

মানে 4টি digit।

23.21 JAVASCRIPT REGEXP

ES2026

```
console.log(/\d{4}/.test("2026"));
```

OUTPUT / TEXT

```
true
```

23.22 {n,m}

Minimum `n`, maximum `m`।

23.22 JAVASCRIPT REGEXP

ES2026

```
/\d{2,4}/
```

মানে 2 থেকে 4 digit।

Match করতে পারে:

OUTPUT / TEXT

```
12
123
1234
```

23.23 {n,}

Minimum `n`।

23.23 JAVASCRIPT REGEXP

ES2026

```
/\d{3,}/
```

মানে অন্তত 3 digit।

23.24 Anchors

Pattern-এর position নির্দিষ্ট করতে anchors ব্যবহার করা হয়।

^

String/line-এর শুরু।

\$

String/line-এর শেষ।

23.25 ^`Example

23.25 JAVASCRIPT REGEXP

ES2026

```
console.log(/^Hello/.test("Hello World"));
```

OUTPUT / TEXT

```
true
```

কিন্তু:

23.25 JAVASCRIPT REGEXP

ES2026

```
console.log(/^Hello/.test("Hi Hello"));
```

OUTPUT / TEXT

```
false
```

কারণ `Hello` শুরুতে নেই।

23.26 `\$`Example

23.26 JAVASCRIPT REGEXP

ES2026

```
console.log(/World$/.test("Hello World"));
```

OUTPUT / TEXT

```
true
```

23.27 ^ + \$

দুটো একসাথে দিলে পুরো string match করার ধারণা পাওয়া যায়।

23.27 JAVASCRIPT REGEXP

ES2026

```
console.log(/^12345$/.test("12345"));
```

OUTPUT / TEXT

```
true
```

কিন্তু:

23.27 JAVASCRIPT REGEXP

ES2026

```
console.log(/^12345$/.test("123a"));
```

OUTPUT / TEXT

```
false
```

কারণ পুরো string-এ শুধু digit নেই।

23.28 Alternation `|`

মানে:

OR

23.28 JAVASCRIPT REGEXP

ES2026

```
/cat|dog/
```

মানে:

OUTPUT / TEXT

cat অথবা dog

Example:

23.28 JAVASCRIPT REGEXP

ES2026

```
console.log(/cat|dog/.test("I have a dog"));
```

OUTPUT / TEXT

true

Part 04 — Groups, Lookaround Assertions & Quantifier Greediness (23.29 – 23.39)

23.29 Grouping `()`

Group তৈরি করতে:

23.29 REGEXP PATTERN

PATTERN

```
( ... )
```

Example:

23.29 JAVASCRIPT REGEXP

ES2026

```
/(cat|dog)/
```

মানে:

OUTPUT / TEXT

cat
অথবা
dog

23.30 Capturing Groups

Regex-এর একটি অংশ আলাদা করে capture করতে পারো।

23.30 JAVASCRIPT REGEXP

ES2026

```
const regex = /(\d{4})-(\d{2})-(\d{2})/;  
const result = "2026-09-16".match(regex);  
console.log(result);
```

Conceptual output:

OUTPUT / TEXT

```
[  
  "2026-09-16",  
  "2026",  
  "09",  
  "16"  
]
```

প্রখানে:

```
Group 1 → 2026  
Group 2 → 09  
Group 3 → 16
```

23.31 Non-Capturing Group

যদি grouping দরকার কিন্তু capture দরকার না হয়:

23.31 REGEXP PATTERN

PATTERN

(?: ...)

Example:

23.31 JAVASCRIPT REGEXP

ES2026

```
const regex = /(?:cat|dog)/;
```

এটি group করবে, কিন্তু capturing group তৈরি করবে না।

23.32 Named Capturing Groups

আরও readable করার জন্য group-এর নাম দেওয়া যায়।

23.32 JAVASCRIPT REGEXP

ES2026

```
const regex =
  /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/;

const result =
  "2026-09-16".match(regex);

console.log(result.groups);
```

23.32 JAVASCRIPT REGEXP

ES2026

```
{
  year: "2026",
  month: "09",
  day: "16"
}
```

এটি advanced কিন্তু real-world parsing-এ useful।

23.33 Backreference

Captured group পরে আবার একই text match করতে ব্যবহার করা যায়।

Example:

23.33 JAVASCRIPT REGEXP

ES2026

```
const regex = /\b(\w+)\s+\1\b/i;

console.log(
  regex.test("hello hello")
);
```

OUTPUT / TEXT

true

এখানে:

OUTPUT / TEXT

(\w+)

প্রথম word capture করেছে।

OUTPUT / TEXT

\1

মানে:

Group 1-এ যে text ছিল, সেটিই আবার আসতে হবে।

23.34 Lookahead

Lookahead pattern-এর পরের অংশ check করে কিন্তু consume করে না।

Positive Lookahead

23.34 REGEXP PATTERN

PATTERN

`X(?:=Y)`

মানে:

X-এর পরে Y থাকতে হবে।

Example:

23.34 JAVASCRIPT REGEXP

ES2026

```
const regex = /\d+(?= USD)/;

console.log(
  "100 USD".match(regex)
);
```

OUTPUT / TEXT

`["100"]`

23.35 Negative Lookahead

23.35 REGEXP PATTERN

PATTERN

`X(?:!Y)`

মানে:

X-এর পরে Y থাকা যাবে না।

Example:

23.35 JAVASCRIPT REGEXP

ES2026

```
const regex = /foo(?:!bar)/;

console.log(regex.test("foobar"));
```

OUTPUT / TEXT

`false`

কিন্তু:

23.35 JAVASCRIPT REGEXP

ES2026

```
console.log(regex.test("foobaz"));
```

OUTPUT / TEXT

`true`

23.36 Lookbehind

Lookbehind সামনে নয়, আগের অংশ check করে।

Positive Lookbehind

23.36 REGEXP PATTERN

PATTERN

`(?<=X)Y`

Example:

23.36 JAVASCRIPT REGEXP

ES2026

```
const regex = /(?<=\$)\d+/;

console.log(
  "$100".match(regex)
);
```

OUTPUT / TEXT

`["100"]`

23.37 Negative Lookbehind

23.37 REGEXP PATTERN

PATTERN

```
(?<!X)Y
```

মানে:

Y-এর আগে X থাকা যাবে না।

Example:

23.37 JAVASCRIPT REGEXP

ES2026

```
const regex = /(?<!\$)\d+;/;

console.log(
  "USD 100".match(regex)
);
```

এখানে `100` match হতে পারে কারণ এর আগে `$` নেই।

23.38 Greedy Matching

Regex quantifier সাধারণত greedy।

মানে:

যত বেশি সম্ভব text match করতে চেষ্টা করে।

Example:

23.38 JAVASCRIPT REGEXP

ES2026

```
const text = "<b>Hello</b><b>World</b>";

console.log(
  text.match(/<b>.*</b>/)
);
```

এখানে `.*` অনেক বেশি text consume করতে পারে।

23.39 Lazy Matching

Lazy করতে `?` ব্যবহার করা যায়।

23.39 JAVASCRIPT REGEXP

ES2026

```
const text = "<b>Hello</b><b>World</b>";

console.log(
  text.match(/<b>.*?</b>/g)
);
```

23.39 JAVASCRIPT REGEXP

ES2026

```
[
  "<b>Hello</b>",
  "<b>World</b>"
]
```

Difference:

```
.* → Greedy
.*? → Lazy
```

23.40 match()

Text-এর মধ্যে match বের করতে:

23.40 JAVASCRIPT REGEXP ES2026

```
const text = "I have 10 apples and 20 oranges.";
const numbers = text.match(/\d+/g);
console.log(numbers);
```

23.40 JAVASCRIPT REGEXP ES2026

["10", "20"]

23.41 matchAll()

Multiple matches এবং capturing groups-এর information নিয়ে কাজ করতে:

23.41 JAVASCRIPT REGEXP ES2026

```
const text = "ID: 101, ID: 202";
const matches =
  [...text.matchAll(/ID: (\d+)/g)];
for (const match of matches) {
  console.log(match[1]);
}
```

OUTPUT / TEXT

101
202

matchAll() iterator return করে।

23.42 search()

প্রথম match-এর index বের করতে:

23.42 JAVASCRIPT REGEXP ES2026

```
const text = "Hello JavaScript";
console.log(
  text.search(/JavaScript/)
);
```

OUTPUT / TEXT

6

কারণ JavaScript index 6 থেকে শুরু।

23.43 replace()

Regex দিয়ে text replace:

23.43 JAVASCRIPT REGEXP ES2026

```
const text = "I like cats.";
const result =
  text.replace(/cats/, "dogs");
console.log(result);
```

OUTPUT / TEXT

I like dogs.

23.44 `replace()` + `g`

23.44 JAVASCRIPT REGEXP

ES2026

```
const text = "cat cat cat";

const result =
  text.replace(/cat/g, "dog");

console.log(result);
```

OUTPUT / TEXT

dog dog dog

23.45 `replaceAll()`

23.45 JAVASCRIPT REGEXP

ES2026

```
const text = "cat cat cat";

const result =
  text.replaceAll("cat", "dog");

console.log(result);
```

OUTPUT / TEXT

dog dog dog

23.46 Replace Callback

Replacement হিসেবে function দেওয়া যায়।

23.46 JAVASCRIPT REGEXP

ES2026

```
const text = "10 20 30";

const result =
  text.replace(/\d+/g, (number) => {

    return Number(number) * 2;

  });

console.log(result);
```

OUTPUT / TEXT

20 40 60

23.47 Email Validation

একটি basic email pattern:

23.47 JAVASCRIPT REGEXP

ES2026

```
const emailRegex =  
  /^[\s@]+@[^\s@]+\.[^\s@]+$/;
```

Test:

23.47 JAVASCRIPT REGEXP

ES2026

```
console.log(  
  emailRegex.test("shariar@example.com")  
);
```

OUTPUT / TEXT

true

Invalid:

23.47 JAVASCRIPT REGEXP

ES2026

```
console.log(  
  emailRegex.test("hello")  
);
```

OUTPUT / TEXT

false

এটি email-এর basic format check; complete RFC-level email validation নয়।

23.48 Phone Number Validation

বাংলাদেশের 11-digit mobile format-এর একটি basic example:

23.48 JAVASCRIPT REGEXP

ES2026

```
const phoneRegex = /^01[3-9]\d{8}$/;  
  
console.log(  
  phoneRegex.test("01712345678")  
);
```

OUTPUT / TEXT

true

এটি format validation-এর example; actual business rules অনুযায়ী validation পরিবর্তন হতে পারে।

23.49 Password Validation

ধরো password-এ থাকতে হবে:

- কমপক্ষে 8 characters
- একটি uppercase
- একটি lowercase
- একটি digit

23.49 JAVASCRIPT REGEXP

ES2026

```
const passwordRegex =  
  /^(?=.*[A-Z])(?=.*[a-z])(?=.*\d){8,}$/;
```

Test:

23.49 JAVASCRIPT REGEXP

ES2026

```
console.log(  
  passwordRegex.test("Hello123")  
);
```

OUTPUT / TEXT

true

23.50 Password Regex Breakdown

23.50 REGEXP PATTERN

PATTERN

^

→ শুরু

23.50 REGEXP PATTERN

PATTERN

(?=.*[A-Z])

→ অন্তত একটি uppercase

23.50 REGEXP PATTERN

PATTERN

(?=.*[a-z])

→ অন্তত একটি lowercase

23.50 REGEXP PATTERN

PATTERN

(?=.*\d)

→ অন্তত একটি digit

23.50 REGEXP PATTERN

PATTERN

{8,}

→ অন্তত 8 characters

23.50 REGEXP PATTERN

PATTERN

\$

→ শেষ

23.51 Stronger Password Example

ধরো:

- 8+ characters
- uppercase
- lowercase
- number
- special character

23.51 JAVASCRIPT REGEXP

ES2026

```
const strongPassword =  
  /^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[^\A-Za-z0-9]).{8,}$;/;  
  
console.log(  
  strongPassword.test("Hello@123")  
);
```

OUTPUT / TEXT

true

23.52 Username Validation

ধরো:

- letters
- numbers
- underscore
- 3–16 characters

23.52 JAVASCRIPT REGEXP

ES2026

```
const usernameRegex =  
  /^[A-Za-z0-9_]{3,16}$/;  
  
console.log(  
  usernameRegex.test("Shariar_01")  
);
```

OUTPUT / TEXT

true

23.53 Extract Numbers

23.53 JAVASCRIPT REGEXP

ES2026

```
const text =  
  "I bought 2 laptops and 3 phones."  
  
const numbers =  
  text.match(/\d+/g)  
  
console.log(numbers);
```

23.53 JAVASCRIPT REGEXP

ES2026

```
["2", "3"]
```

23.54 Extract Hashtags

23.54 JAVASCRIPT REGEXP

ES2026

```
const text =  
  "Learning #JavaScript and #React today."  
  
const hashtags =  
  text.match(/#[A-Za-z0-9_]+/g)  
  
console.log(hashtags);
```

23.54 JAVASCRIPT REGEXP

ES2026

```
[  
  "#JavaScript",  
  "#React"  
]
```

23.55 Remove Extra Spaces

23.55 JAVASCRIPT REGEXP

ES2026

```
const text =  
  "Hello    JavaScript    World"  
  
const cleaned =  
  text.replace(/\s+/g, " ")  
  
console.log(cleaned);
```

OUTPUT / TEXT

```
Hello JavaScript World
```

23.56 Remove Special Characters

23.56 JAVASCRIPT REGEXP

ES2026

```
const text =  
  "Hello! @Shariar #Developer"  
  
const cleaned =  
  text.replace(/[^\\w\\s]/g, "")  
  
console.log(cleaned);
```

OUTPUT / TEXT

```
Hello Shariar Developer
```

23.57 Extract URLs

একটি basic URL pattern:

```
23.57 JAVASCRIPT REGEXP ES2026

const urlRegex =
  /https?:\/\/[^\s]+/g;

const text =
  "Visit https://example.com for more info.";

console.log(
  text.match(urlRegex)
);
```

```
23.57 JAVASCRIPT REGEXP ES2026

[
  "https://example.com"
]
```

বাস্তব URL parsing-এর জন্য [URL](#) object অনেক ক্ষেত্রে Regex-এর চেয়ে ভালো।

23.58 Regex দিয়ে Date Extract

```
23.58 JAVASCRIPT REGEXP ES2026

const text =
  "Exam date: 2026-09-20";

const dateRegex =
  /\d{4}-\d{2}-\d{2}/;

console.log(
  text.match(dateRegex)
);
```

```
23.58 JAVASCRIPT REGEXP ES2026

["2026-09-20"]
```

23.59 Date Validation-এর সীমাবদ্ধতা

এই Regex:

```
23.59 REGEXP PATTERN PATTERN

\d{4}-\d{2}-\d{2}
```

এটি:

```
OUTPUT / TEXT

2026-99-99
```

এর মতো format-ও match করতে পারে।

অর্থাৎ:

Regex format check করতে পারে, কিন্তু সবসময় date-এর বাস্তব validity নিশ্চিত করে না।

তাই:

```
23.59 JAVASCRIPT REGEXP ES2026

Date
```

দিয়ে additional validation লাগতে পারে।

23.60 Regex Escape Characters

Regex-এ কিছু character-এর special meaning আছে:

OUTPUT / TEXT

. * + ? ^ \$ { } [] () | \

Literal হিসেবে ব্যবহার করতে escape করতে হয়।

Example:

23.60 JAVASCRIPT REGEXP

ES2026

```
const regex = /\./;
```

এখানে `.`-কে literal dot হিসেবে match করা হচ্ছে।

23.61 Literal Dot

23.61 JAVASCRIPT REGEXP

ES2026

```
console.log(/\./test("example.com"));
```

OUTPUT / TEXT

true

23.62 Literal ``$``

Dollar sign match করতে:

23.62 JAVASCRIPT REGEXP

ES2026

```
/\$/
```

Example:

23.62 JAVASCRIPT REGEXP

ES2026

```
console.log(/\$100/.test("$100"));
```

OUTPUT / TEXT

true

23.63 Word Boundary ``\b``

`\b`` word boundary নির্দেশ করে।

23.63 JAVASCRIPT REGEXP

ES2026

```
const regex = /\bcat\b/;
```

```
console.log(regex.test("cat"));
```

true

কিন্তু:

23.63 JAVASCRIPT REGEXP

ES2026

```
console.log(regex.test("category"));
```

false হতে পারে, কারণ `cat`` এখানে পুরো word নয়।

23.64 ``\B``

Non-word-boundary।

এটি তুলনামূলকভাবে advanced এবং কম ব্যবহৃত।

23.65 Multiline `m`

`m` flag ব্যবহার করলে `^` এবং `$` প্রতিটি line-এর beginning/end-এর সাথে কাজ করতে পারে।

23.65 JAVASCRIPT REGEXP

ES2026

```
const text = `Hello
World`;

console.log(
  text.match(/^World$/m)
);
```

23.65 JAVASCRIPT REGEXP

ES2026

```
["World"]
```

23.66 Unicode `u`

Unicode-aware matching-এর জন্য `u` flag ব্যবহার করা যায়।

23.66 JAVASCRIPT REGEXP

ES2026

```
const regex = /\u{1F600}/u;

console.log(
  regex.test("😄")
);
```

OUTPUT / TEXT

```
true
```

Unicode নিয়ে কাজ করার সময় `u` এবং Unicode property escapes useful।

23.67 Unicode Property Escapes

Modern JavaScript-এ:

23.67 REGEXP PATTERN

PATTERN

```
\p{ ... }
```

ব্যবহার করা যায় `u` flag-এর সাথে।

Example:

23.67 JAVASCRIPT REGEXP

ES2026

```
const regex = /\p{L}+/gu;

console.log(
  "Hello বাংলা".match(regex)
);
```

Output conceptually:

23.67 JAVASCRIPT REGEXP

ES2026

```
[
  "Hello",
  "বাংলা"
]
```

এখানে `\p{L}` letter-এর Unicode property বোঝায়।

23.68 Emoji Matching

Unicode property দিয়ে emoji detect করা সম্ভব:

23.68 JAVASCRIPT REGEXP

ES2026

```
const emojiRegex =
  /\p{Emoji}/u;

console.log(
  emojiRegex.test("😄")
);
```

OUTPUT / TEXT

```
true
```

23.69 Sticky y Flag

y flag match-কে lastIndex position-এ stick করে।

Example:

23.69 JAVASCRIPT REGEXP

ES2026

```
const regex = /\d/y;

regex.lastIndex = 0;

console.log(
  regex.test("123")
);
```

OUTPUT / TEXT

true

y advanced parsing scenarios-এ বেশি useful।

23.70 lastIndex

g এবং y flag-এর সাথে lastIndex গুরুত্বপূর্ণ হতে পারে।

23.70 JAVASCRIPT REGEXP

ES2026

```
const regex = /\d/g;

console.log(regex.exec("123"));
console.log(regex.exec("123"));
```

Result sequentially match করতে পারে।

Conceptually:

OUTPUT / TEXT

1
2

এখানে regex-এর internal position পরিবর্তিত হয়।

23.71 exec()

23.71 JAVASCRIPT REGEXP

ES2026

```
const regex = /\d+/;

const result =
  regex.exec("Age: 24");

console.log(result);
```

Output conceptually:

23.71 JAVASCRIPT REGEXP

ES2026

```
[
  "24",
  index: 5,
  input: "Age: 24"
]
```

exec() match-এর detailed result দেয়।

23.72 test() বনাম match()

Method	কাজ
test()	match আছে কি না
match()	matched text বের করা
matchAll()	সব match + groups
search()	প্রথম match-এর index
replace()	text replace
exec()	detailed match result

23.73 Regex + Form Validation

ধরো:

23.73 HTML

MARKUP

```
<input id="email" type="text">
<button id="submit">Submit</button>
```

JavaScript:

23.73 JAVASCRIPT REGEXP

ES2026

```
const emailInput =
  document.querySelector("#email");

const emailRegex =
  /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

document
  .querySelector("#submit")
  .addEventListener("click", () => {

    if (emailRegex.test(emailInput.value)) {

      console.log("Valid email");

    } else {

      console.log("Invalid email");

    }

  });
```

23.74 Regex + Input Sanitization

Regex দিয়ে unwanted characters remove করা যায়।

23.74 JAVASCRIPT REGEXP

ES2026

```
const username = "Shariar@123!";

const cleaned =
  username.replace(/^[A-Za-z0-9_]/g, "");

console.log(cleaned);
```

OUTPUT / TEXT

Shariar123

কিন্তু মনে রাখবে:

Sanitization এবং validation একই জিনিস নয়।

23.75 Validation vs Sanitization

Validation

Check করে:

Data valid কি না?

23.75 JAVASCRIPT REGEXP

ES2026

```
/^[A-Za-z]+$//
```

Sanitization

Data clean/transform করে:

23.75 JAVASCRIPT REGEXP

ES2026

```
text.replace( ... )
```

23.76 Regex Security

Regex-এর ক্ষেত্রে একটি advanced security issue আছে:

⚠️ ReDoS

Regular Expression Denial of Service

কিছু poorly designed regex খুব complex input-এর ক্ষেত্রে অত্যন্ত slow হয়ে যেতে পারে।

বিশেষ করে nested quantifier-এর মতো dangerous pattern:

23.76 REGEXP PATTERN

PATTERN

`(a+)+`

এর মতো pattern carefully ব্যবহার করতে হয়।

Best Practice

- Unnecessary complex regex avoid করো
- User-controlled huge input limit করো
- Nested quantifier সাবধানে ব্যবহার করো
- Performance-sensitive regex test করো

23.77 Regex সব সমস্যার সমাধান নয়

Regex দিয়ে:

```
Email format
Phone format
Username
Password rules
Text extraction
Search
Replace
```

খুব ভালোভাবে করা যায়।

কিন্তু complex structured data-এর জন্য অন্য tools ভালো হতে পারে।

উদাহরণ:

URL

Regex-এর বদলে:

23.77 JAVASCRIPT REGEXP

ES2026

`new URL(url)`

Date

Regex +:

23.77 JAVASCRIPT REGEXP

ES2026

`Date`

HTML parsing

Regex দিয়ে HTML parser replace করা উচিত নয়।

23.78 Real-Life Project — Registration Validator

23.78 JAVASCRIPT REGEXP

ES2026

```
const usernameRegex =
  /^[A-Za-z0-9_]{3,16}$/;

const emailRegex =
  /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

const phoneRegex =
  /^01[3-9]\d{8}$/;

const passwordRegex =
  /^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[A-Za-z0-9]).{8,}$/;

function validateUser(user) {

  if (!usernameRegex.test(user.username)) {
    return "Invalid username";
  }

  if (!emailRegex.test(user.email)) {
    return "Invalid email";
  }

  if (!phoneRegex.test(user.phone)) {
    return "Invalid phone";
  }

  if (!passwordRegex.test(user.password)) {
    return "Weak password";
  }

  return "Registration data is valid";
}

const user = {

  username: "Shariar_01",

  email: "shariar@example.com",

  phone: "01712345678",

  password: "Hello@123"

};

console.log(validateUser(user));
```

CONSOLE OUTPUT

Registration data is valid

23.79 Regex-এর একটি Practical Text Processing Example

ধরো server থেকে পেলাম:

23.79 JAVASCRIPT REGEXP

ES2026

```
const text = `
Name: Shariar
Age: 24
Email: shariar@example.com
Phone: 01712345678
`;
```

Email বের করি:

23.79 JAVASCRIPT REGEXP

ES2026

```
const email =
  text.match(
    /[\s@]+@[^\s@]+\.[^\s@]+/
  );

console.log(email[0]);
```

OUTPUT / TEXT

shariar@example.com

Phone:

23.79 JAVASCRIPT REGEXP

ES2026

```
const phone =
  text.match(/01[3-9]\d{8}/);

console.log(phone[0]);
```

OUTPUT / TEXT

01712345678

23.80 Regex Pattern পড়ার কৌশল

যখন একটি বড় Regex দেখবে, পুরোটা একসাথে বোঝার চেষ্টা করবে না।

ধরো:

23.80 REGEXP PATTERN

PATTERN

```
^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[A-Za-z0-9]).{8,}$
```

এটাকে ভাঙে:

```
^
↓
start

(?=.*[A-Z])
↓
uppercase আছে?

(?=.*[a-z])
↓
lowercase আছে?

(?=.*\d)
↓
digit আছে?

(?=.*[A-Za-z0-9])
↓
special character আছে?

.{8,}
↓
minimum 8 characters

$
↓
end
```

এভাবেই Regex পড়তে হয়।

MUST KNOW 🔥 23.81 Must Know — Advanced Regular Expressions Pillars**Basic**

- Regular Expression
- Regex literal
- `RegExp`
- `test()`
- Regex flags

⚙️ **Character Classes**

OUTPUT / TEXT

```
[abc]
[a-z]
[A-Z]
[0-9]
[^abc]
\d
\D
\w
\W
\s
\S
.
```

Quantifiers

OUTPUT / TEXT

```
+
*
?
{n}
{n,m}
{n,}
```

Anchors

OUTPUT / TEXT

```
^
$
\b
\B
```

Groups

OUTPUT / TEXT

```
( ... )
(?: ... )
(?<name> ... )
```

Advanced

OUTPUT / TEXT

```
\1
(?:= ... )
(?:! ... )
(?:<= ... )
(?:<! ... )
```

⚙️ **Methods**

JAVASCRIPT REGEXP

ES2026

```
test()
exec()
match()
matchAll()
search()
replace()
replaceAll()
```

Practical

- Email validation
- Phone validation
- Password validation
- Username validation
- Number extraction

- Hashtag extraction
- URL extraction
- Text cleaning

Advanced Topics

- Greedy vs lazy
- Unicode
- Unicode property escapes
- `g`, `m`, `s`, `u`, `y`
- `lastIndex`
- ReDoS
- Regex limitations

CHEAT SHEET 🗨 Quick Cheat Sheet — RegEx Syntax, Quantifiers & Lookarounds

REGEXP PATTERN

PATTERN

/abc/

→ abc

REGEXP PATTERN

PATTERN

/abc/i

→ case-insensitive

REGEXP PATTERN

PATTERN

/abc/g

→ global

REGEXP PATTERN

PATTERN

\d

→ digit

REGEXP PATTERN

PATTERN

\w

→ word character

REGEXP PATTERN

PATTERN

\s

→ whitespace

REGEXP PATTERN

PATTERN

.

→ any character except newline by default

REGEXP PATTERN

PATTERN

+

→ 1 or more

REGEXP PATTERN

PATTERN

*

→ 0 or more

REGEXP PATTERN

PATTERN

?

→ 0 or 1

REGEXP PATTERN

PATTERN

{3}

→ exactly 3

REGEXP PATTERN

PATTERN

{2,5}

→ 2–5

REGEXP PATTERN

PATTERN

^

→ start

REGEXP PATTERN

PATTERN

\$

→ end

REGEXP PATTERN

PATTERN

|

→ OR

REGEXP PATTERN

PATTERN

(...)

→ capturing group

REGEXP PATTERN

PATTERN

(?: ...)

→ non-capturing group

REGEXP PATTERN

PATTERN

(?= ...)

→ positive lookahead

REGEXP PATTERN

PATTERN

(?! ...)

→ negative lookahead

REGEXP PATTERN

PATTERN

(?<= ...)

→ positive lookbehind

REGEXP PATTERN

PATTERN

(?<!= ...)

→ negative lookbehind

REGEXP PATTERN

PATTERN

\1

→ first captured group-এর backreference



Practice 1

এমন Regex বানাও যেটা শুধু digit accept করবে:

```
12345 → true
123a5 → false
```

Practice 2

এমন Regex বানাও যেটা username:

OUTPUT / TEXT

```
3-15 characters
letters
numbers
underscore
```

accept করবে।

Practice 3

এই text থেকে সব number বের করো:

JAVASCRIPT REGEXP

ES2026

```
const text =
  "I have 10 books, 20 pens and 5 laptops.";
```

Expected:

OUTPUT / TEXT

```
["10", "20", "5"]
```

Practice 4

এই text থেকে সব hashtag বের করো:

JAVASCRIPT REGEXP

ES2026

```
const text =
  "I love #JavaScript #React #NodeJS";
```

Expected:

OUTPUT / TEXT

```
#JavaScript
#React
#NodeJS
```

Practice 5

Extra spaces remove করো:

OUTPUT / TEXT

```
"Hello    World    JavaScript"
```

Expected:

OUTPUT / TEXT

```
"Hello World JavaScript"
```

Practice 6

একটি password validator বানাও যেখানে:

- minimum 8 characters
- uppercase
- lowercase
- number
- special character

থাকবে।

Practice 7 — Advanced

এই string:

OUTPUT / TEXT

```
"ID: 101, ID: 202, ID: 303"
```

থেকে `matchAll()` ব্যবহার করে:

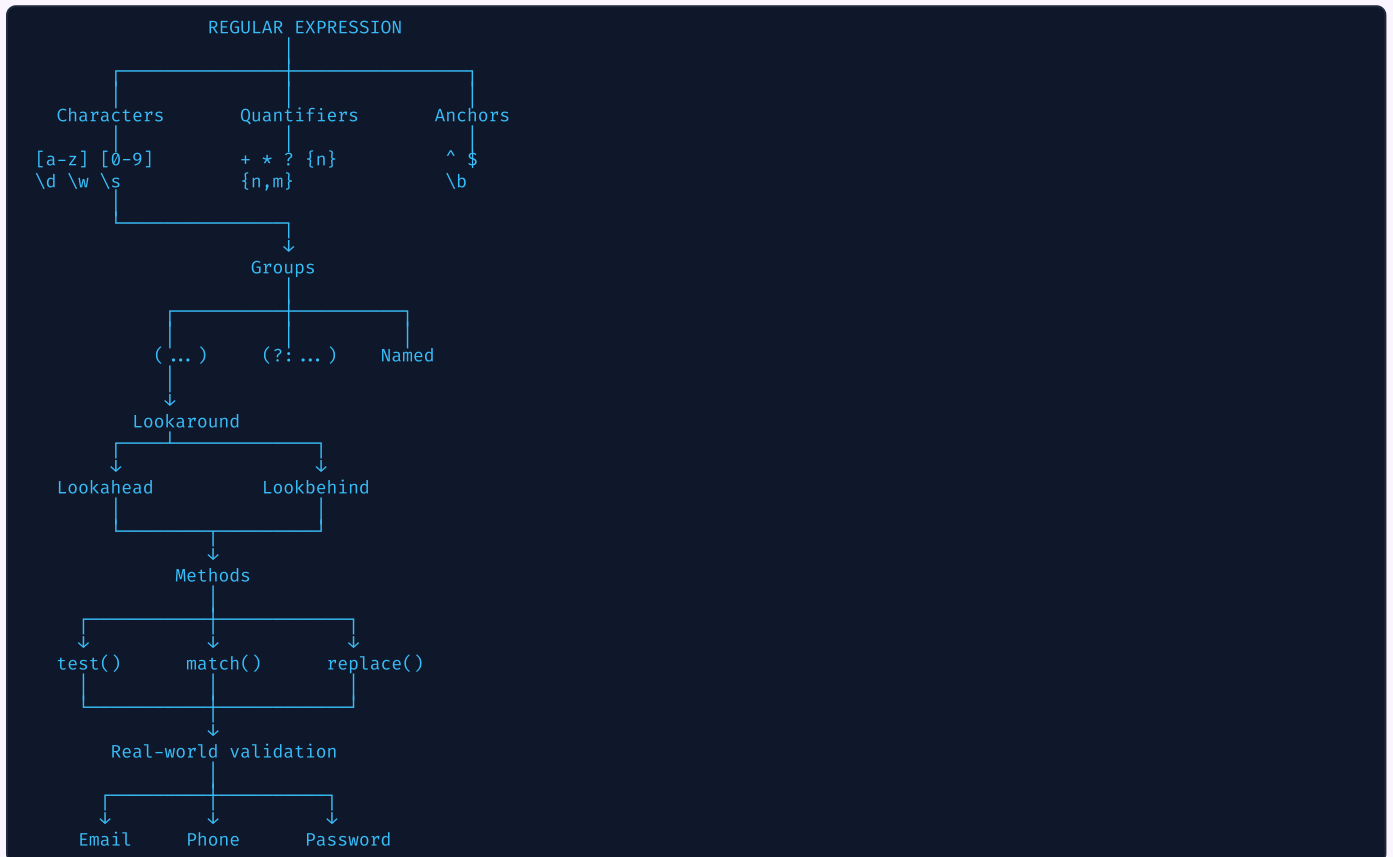
OUTPUT / TEXT

```
101  
202  
303
```

বের করো।

SUMMARY

Chapter 23 Final Mental Map — RegExp Architecture & Use Cases



✦ ☆ সবচেয়ে গুরুত্বপূর্ণ কথা

Regex-কে শুধু **validation tool** হিসেবে দেখবে না।

একজন Full Stack Developer হিসেবে Regex ব্যবহার করতে পারবে:

```
User Input  
↓  
Validation  
↓  
Text Search  
↓  
Data Extraction  
↓  
Text Replacement  
↓  
Data Cleaning  
↓  
Parsing
```

আর **Chapter 13-এর basic RegExp + Chapter 23-এর Advanced RegExp** মিলিয়ে এখন JavaScript-এর Regular Expression অংশটি অনেক বেশি complete হলো।

NEXT CHAPTER

Chapter 24 Preview — Object-Oriented JavaScript & Classes

পরবর্তী অধ্যায়ে আমরা জাভাস্ক্রিপ্টের অন্যতম কোর পিলার **Object-Oriented Programming (OOP) & ES6 Classes** নিয়ে বিস্তারিত শিখব— Constructor, Prototypal Inheritance, `extends` & `super`, Private Fields (`#`), Static Methods এবং বাস্তব সফটওয়্যার আর্কিটেকচার ডিজাইন প্যাটার্ন।